Unidad 2 / Escenario 4

Lectura fundamental

Arreglos de tamaño fijo

Contenido

1 Declaración e inicialización 1

2 Recorrido de arreglos 2

3 Utilidades estándar 3

4 Cálculo de la desviación estándar 4

5 Arreglos de dos dimensiones 5

6 Arreglos de más de dos dimensiones 8

7 Optimización de recursiones 9

8 Problemas propuestos 12

Palabras clave: arreglos, cálculo de desviación, arreglos y matrices.

Los arreglos de datos de tamaño fijo fueron las primeras herramientas para almacenar grandes conjuntos de datos que se podían referenciar a partir de una sola variable y un índice (el cual generalmente va entre paréntesis cuadrados, como es el caso de Java). Este índice en memoria física del computador significa un corrimiento en bits desde la posición inicial desde donde está almacenada la variable. Su uso es muy importante puesto que proporciona maneras sencillas de almacenar varios (posiblemente muchos) valores del mismo tipo de dato, de tal manera que su consulta (acceso) es muy eficiente.

1. Declaración e inicialización

Tal como con los tipos de datos nativos, la declaración de arreglos de tamaño fijo tiene ciertas reglas según el lenguaje. La forma de declarar arreglos depende de cómo se vayan a usar. Para mayor entendimiento de este tema, se recomienda en todo lenguaje revisar la documentación. En el caso de Java, se sugiere ver Oracle (1995). La preparación para el uso de arreglos se puede dividir en dos. Primero, la declaración de la variable, que es como se puede ver en el Código 1 para **x** y **b**; por ejemplo, **int x[]** y **int b[][]**. Estas instrucciones informan al compilador que habrá una referencia a un arreglo de cierto tipo de dato; en el ejemplo, enteros. La segunda manera es la instanciación, que consiste en informar al compilador la cantidad de memoria que se necesita; se puede identificar fácilmente porque requiere del uso de la palabra reservada *new*. Ambas cosas se pueden hacer en una sola línea, como se puede ver también en el Código 1: **int x[] = new int[n]**; y **int a[][] = new int[n][m]**; declaran e inicializan un arreglo llamado **x** de tamaño dado por el valor entero *n* y un arreglo de dos dimensiones llamado **a** de dimensiones *n* filas por *m* columnas.

En el Código 1 se pueden ver ejemplos de declaración de dos arreglos de tamaño fijo de una dimensión; el arreglo **x** y el arreglo **y**, ambos de tamaño ***n***. También se aprecian tres formas de declarar arreglos de dos dimensiones de tamaño *n* filas por *m* columnas. Adicional a estas formas de declarar e inicializar arreglos, hay otras. Una variable que puede haber en los arreglos de más de una dimensión es que, por ejemplo, en los arreglos de dos dimensiones cada fila puede tener diferentes posiciones a las otras.

```java
import java.util.Arrays;
public class ArraysDeclarations{
    public static void main (String[] args){

        int n = 3, m = 2;

        int x[] = new int[n];

        int y[];
        y = new int[n];

        int a[][] = new int[n][m];

        int b[][];
        b = new int[n][];

        for( int[] arr: b )
            arr = new int[m];

        int c[][];
        c = new int[n][];
        for( int i = 0; i < c.length; ++ i )
            c[i] = new int[m];
    }
}
```

Código 1: declaración de arreglos.

2. Recorrido de arreglos

Los arreglos se pueden recorrer de diferentes maneras. Las más comunes son el uso de la estructura *for each* y el uso de un ciclo *for* con una variable tipo contador a usar como índice. En el Código 3 hay ejemplos de cómo declarar y recorrer arreglos. Como siempre, se recomienda compilar, ejecutar y seguir paso a paso el código para entender la manera en que está llenado el arreglo **a** y la matriz **b**.

```
import java.util.Arrays; public class
ArraysIteration{
    public static void main (String[] args){
        int n = 5;
        int a[] = new int[n];
        for (int i = 0; i < a.length; i++)
                a[i] = i*i*i; for( int x : a )
            System.out.print(x +" ");

        int m = 4;
        int b[][] = new int[n][m];
        for (int i = 0; i < b.length; i++)
            for (int j = 0; j < b[i].length; j++)
                b[i][j] = i*j;
        for( int row[] : b )
        {
            for( int x: row)
                System.out.print(x +" ");
            System.out.println();
        }
    }
}
```

Código 2: recorrido de arreglos.

3. Utilidades estándar

En los lenguajes de programación de propósito general, hay rutinas que permiten hacer ciertas tareas con arreglos de una manera corta o eficiente. En el caso de Java, entre otros, están los métodos en la clase *Arrays* y la clase *System*:

- *Arrays.toString()*: sirve para obtener la representación en cadena de un arreglo de una dimensión.
- *Arrays.deepTostring()*: de manera similar al anterior, se usa para obtener la representación en cadena de un arreglo de varias dimensiones.
- *System.arraycopy()*: sirve para copiar el contenido de un arreglo a otro. Recibe varios parámetros; para más detalles, revisar la documentación de Java.
- *Arrays.sort()*: ordena el contenido de un arreglo de manera ascendente.
- *Arrays.equals()*: se utiliza para comparar el contenido de dos arreglos de una dimensión elemento a elemento. Retorna true si son iguales.
- *Arrays.deepEquals()*: de manera similar al anterior, sirve para comparar valor a valor dos arreglos de varias dimensiones.

Es necesario analizar el código mostrado en los ejemplos. Se recomienda hacer su seguimiento paso a paso y comprobar que el análisis produce los resultados predichos y verificarlos con la ejecución del programa; también se sugiere introducir cambios en el código mostrado y predecir los cambios que tendrá la salida o el programa en su compilación y ejecución.

```
import java.util.Arrays;
public class ArraysIteration{

    public static void main (String[] args){
        int n = 5;
        int a[] = new int[n];
        for (int i = 0; i < a.length; i++)
                a[i] = (n-i-1)*(n-i-1)*(n-i-1);
            System.out.println(Arrays.toString(a));

        int m = 4;
        int b[][] = new int[n][m];
        for (int i = 0; i < b.length; i++)
            for (int j = 0; j < b[i].length; j++)
                b[i][j] = i*j;
        System.out.println(Arrays.deepToString(b));

        int c[] = new int[n];
        System.arraycopy(a, 0, c, 0, n);
        System.out.println(Arrays.equals(a, c));
        int d[][] = new int[n][m];
        int i = 0;
        for( int arr[] : b)
                System.arraycopy(arr, 0, d[i++], 0, m);
        System.out.println(Arrays.deepEquals(b, d));

        Arrays.sort(a);
        System.out.println(Arrays.toString(a));
    }
}
```

Código 3: utilidades arreglos.

4. Cálculo de la desviación estándar

Como ejemplo de uso de arreglos, consideremos la tarea de calcular una medida estadística, como la desviación estándar de una muestra de valores numéricos. La desviación estándar está dada por:

$$s = \sqrt[2]{\frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n-1}}$$

En donde n es la cantidad de valores, $\bar{x}$ es el promedio de los n valores, y cada x_i un valor. En el Código 4 se puede ver un programa que calcula la desviación estándar de un conjunto de n valores. El programa primero solicita la cantidad de valores, la cual es un entero, y a continuación crea un arreglo de *doubles* de tamaño n para almacenar los n valores. Después de esto, usa los dos métodos creados para calcular la desviación estándar; el método que obtiene el promedio ($\bar{x}$) es usado dentro del método que calcula la desviación estándar.

```
import java.util.Scanner;
public class SqrtRecursion
{
    public static void main(String[] args){
        Scanner sn = new Scanner( System.in );
        System.out.print("How many values?: ");
        int n = sn.nextInt();
        double arr[] = new double[n]; System.out.print("Type
        the "+n+" values: ");
        for (int i = 0; i < arr.length; i++)
            arr[i] = sn.nextDouble();
        double sd = getStandardDeviation(arr);
        System.out.println("Standard Deviation: "+sd);
    }
    public static double getStandardDeviation( double x[] )
    {
        double answer = 0;
        double average = getAverage(x);
        for (int i = 0; i < x.length; i++)
          answer += Math.pow( x[i] - average , 2 ); return
                        Math.sqrt( answer / (x.length-1) );
    }
    public static double getAverage( double x[] )
    {
        double answer = 0;
        for( double a: x )
                answer += a;
          return answer / x.length;
    }
}
```

Código 4: cálculo de desviación estándar.

5. Arreglos de dos dimensiones

Una forma comúnmente usada para almacenar datos consiste en colocarlos como tabla. A estos arreglos de dos dimensiones se accede por medio de dos índices, los cuales, en el caso de Java y otros lenguajes, van entre paréntesis cuadrados. Esto, en muchos lenguajes de programación, se puede hacer creando arreglos de dos dimensiones. La declaración y uso de estos arreglos se pueden ver en los códigos 5 y 6.

En este programa se diseñaron e implementaron funciones para leer una matriz rectangular de la entrada estándar, un método para imprimir a la salida estándar una matriz rectangular, y un método para sumar dos matrices rectangulares posición a posición, siempre y cuando sus tamaños lo permitan; es decir, siempre y cuando los tamaños sean los mismos (la misma cantidad de filas y de columnas).

```
import java.util.Arrays;
import java.util.Scanner;
public class MatrixAddition
{
    private static Scanner sn = new Scanner( System.in ); public
    static void main(String[] args)
    {
        int matrixA[][] = readMatrix(); int
        matrixB[][] = readMatrix();

        System.out.println("Values: "+Arrays.deepToString(matrixA));
        System.out.println("Using a custom method: ");
        System.out.println(" A = ");
        printMatrix(matrixA);
        System.out.println(" B = ");
        printMatrix(matrixB);
        int matrixC[][] = add(matrixA,matrixB);
        if( matrixC == null )
            System.err.println("Error: Different sizes.");
        else
        {
            System.out.println("C = ");
            printMatrix(matrixC);
        }
    }
```

Código 5: suma de dos matrices rectangulares 1/2.

```
    private static int[][] add(int[][] a, int[][] b) {
        if( a.length != b.length ||
            a[0].length != b[0].length ) return null;
        int r = a.length;
        int c = a[0].length;
        int matrix[][] = new int[r][c];
        for (int i = 0; i < matrix.length; i++)
            for (int j = 0; j < matrix[0].length; j++)
                matrix[i][j] = a[i][j]+b[i][j]; return
        matrix;
    }
    private static int[][] readMatrix() {
        System.out.print("How many rows?: ");
        int r = sn.nextInt();
        System.out.print("How many columns?: ");
        int c = sn.nextInt();
        int matrix[][] = new int[r][c];
        for (int i = 0; i < matrix.length; i++)
            for (int j = 0; j < matrix[0].length; j++)
            {
                System.out.print("Type value at position "+i+","+j+" : "); matrix[i][j]
                = sn.nextInt();
            }
        return matrix;
    }
    public static void printMatrix( int a[][] ){
        System.out.println("Matrix "); for (int i
        = 0; i < a.length; i++)
        {
            for (int j = 0; j < a[i].length; j++)
                System.out.print(a[i][j]+"\t");
            System.out.println();
        }
    }
}
```

Código 6: suma de dos matrices rectangulares 2/2.

No obstante, los usos más comunes de arreglos de dos dimensiones implican abstracciones de tablas o matrices en las que la cantidad de posiciones por cada fila es la misma. Java y muchos lenguajes de programación permiten crear arreglos en donde cada posición es un arreglo de tamaño posiblemente diferente. Tal posibilidad la podemos ver en el Código 7, en donde se muestra un programa que lee un arreglo de arreglos; es decir, lee una matriz en donde cada fila puede tener diferente cantidad de elementos a las demás.

```
import java.util.Scanner; public
class ArrayOfArrays{
    private static Scanner sn = new Scanner( System.in ); public
    static void main(String[] args){
        int x[][] = readArrayOfArrays();
        printMatrix(x);
    }
    private static int[][] readArrayOfArrays() {
        System.out.print("How many rows?: ");
        int r = sn.nextInt();
        int matrix[][] = new int[r][];
        for (int i = 0; i < matrix.length; i++){
            System.out.print("How many positions for row "+i+"? : "); int c =
            sn.nextInt();
            matrix[i] = new int[c];
            for (int j = 0; j < matrix[i].length; j++){
                System.out.print("Type value at position "+i+","+j+" : "); matrix[i][j]
                = sn.nextInt();
            }
        }
        return matrix;
    }
    public static void printMatrix( int a[][] ){
        System.out.println("Matrix ");
        for (int i = 0; i < a.length; i++){
            for (int j = 0; j < a[i].length; j++)
                System.out.print(a[i][j]+"\t");
            System.out.println();
        }
    }
}
```

Código 7: arreglo de arreglos.

6. Arreglos de más de dos dimensiones

Aunque no es muy común en la práctica, es posible en Java y en muchos lenguajes crear arreglos de más de dos dimensiones a los cuales se accede con más de dos índices. El número de dimensiones en un arreglo depende de la implementación de Java que se tenga; normalmente hay un tamaño máximo de 255 dimensiones. Un ejemplo de su uso puede ser el modelamiento de un espacio en tres dimensiones, en donde para cada coordenada (x, y, z) se debe tener una medida de temperatura. En el Código 8 se muestra un ejemplo de uso de arreglo de tres dimensiones. Por simplicidad y para facilitar el análisis del código al ejecutarlo, este programa solo solicita datos de las tres dimensiones y llena aleatoriamente el arreglo de tres dimensiones.

```
import java.util.Arrays;
import java.util.Scanner;
public class ThreeDimensionsArray {
    public static void main(String[] args) {
        Scanner sn = new Scanner( System.in );
        System.out.print("Type the X dimension: ");
        int xDim = sn.nextInt();
        System.out.print("Type the Y dimension: ");
        int yDim = sn.nextInt();
        System.out.print("Type the Z dimension: ");
        int zDim = sn.nextInt();
        int roomTemperature[][][] = new int[xDim][yDim][zDim];
        randomFill(roomTemperature);
        System.out.println(Arrays.deepToString(roomTemperature));
        print3DMatrix(roomTemperature);
    }
    private static void print3DMatrix( int [][][] a) {
        System.out.println("3DMatrix: ");
        for (int i = 0; i < a.length; i++) {
            System.out.println("Level "+i+" : ");
            for (int j = 0; j < a[i].length; j++){
                for (int k = 0; k < a[i][j].length; k++)
                    System.out.print(a[i][j][k]+"\t");
                System.out.println();
            }
        }
    }
    private static void randomFill( int [][][] a) {
        for (int i = 0; i < a.length; i++)
            for (int j = 0; j < a[i].length; j++)
                for (int k = 0; k < a[i][j].length; k++)
                    a[i][j][k] = (int)(Math.random()*100);
    }
}
```

Código 8: ejemplo de uso de arreglo de 3 dimensiones.

7. Optimización de recursiones

En algunos problemas de naturaleza recursiva es posible que las funciones o métodos que se diseñen tengan un comportamiento que se degrada muy rápidamente a medida que los valores de sus argumentos crecen. La función combinatorio es un buen ejemplo de ello (también lo es Fibonacci). Cuando en una función recursiva hay soluciones a subproblemas que se solapan, es posible, con unos pequeños cambios, lograr una mayor eficiencia.

Para esto se debe tener una estructura de almacenamiento de datos de subproblemas. En el Código 9 se puede ver dicha optimización sobre la función puramente recursiva mostrada en el Escenario anterior. Para ello se crea un arreglo de dos dimensiones *memoC* justo antes de los ciclos que calculan todos los valores del combinatorio entre (0, 0) y (*MAX* − 1, *MAX* − 1). Para propósitos de demostración, solo se están sumando esos valores: en la variable *sum1* para el cálculo puramente recursivo, y en la variable *sum2* para el cálculo optimizado. Para más detalles en esta técnica, conocida algunas veces como ***memoization*** (si, sin la r), se pueden ver capítulos de programación dinámica en libros de análisis y diseño de algoritmos como el de Cormen, Leiserson, Rivest y Stein (2009) y el de Guerequeta y Vallecillo (2000).

```
import java.util.Arrays;
public class RecursionOptimization {
    private static final int MAX = 30; private
    static int[][] memoC;
    private static final int MAX_WIDTH = 80;
    public static void main(String[] args){
        long init1 = System.currentTimeMillis();
        int sum1 = 0;
        for( int i = 0; i < MAX; ++ i )
            for( int j = 0; j <= i; ++j )
                sum1 += c(i,j);
        long end1 = System.currentTimeMillis();
        System.out.println((end1-init1)+" milliseconds for c(n,k)");

        long init2 = System.currentTimeMillis();
        memoC = new int[MAX][MAX];
        for (int[] arr : memoC)
            Arrays.fill(arr, -1);
        int sum2 = 0;
        for( int i = 0; i < MAX; ++ i )
            for( int j = 0; j <= i; ++j )
                sum2 += c2(i,j);
        long end2 = System.currentTimeMillis();
        System.out.println((end2-init2)+" milliseconds for c2(n,k)");
        System.out.println(sum1+" = "+sum2);
    }
    private static int c2(int n, int k) {
        if( k > n )
            return 0;
        if( k == n || k == 0 )
            return 1;
        if( memoC[n][k] != -1 )
            return memoC[n][k];
        return memoC[n][k] = c2(n-1,k-1) + c2(n-1,k);
    }
    public static int c( int n , int k ){
        if( k > n )
            return 0;
        if( k == n || k == 0 )
            return 1;
        return c(n-1,k-1) + c(n-1,k);
    }
}
```

Código 9: verificación de optimización del combinatorio.

8. Problemas propuestos

Casi todos los problemas de esta sección constan de un enunciado en el que se solicita el diseño e implementación de un programa en Java que tome como entrada ciertos datos leídos desde la entrada estándar de este lenguaje y produzca como única salida los datos solicitados, que pueden ser cálculos o transformaciones hechas con los datos de entrada. En todos los problemas propuestos se debe asumir que las entradas cumplirán con lo estipulado en la sección "`Descripción de entrada`" de cada problema. Dicho esto, se debe entonces diseñar e implementar un programa que cumpla con lo estipulado en la sección "`Descripción de salida`", y que sea únicamente lo solicitado allí, nada más.

Teniendo en cuenta que en este **Escenario se estudiaron los arreglos de tamaño fijo de una o más dimensiones**, se espera que en todas las soluciones se usen de alguna manera. Dado que en el Escenario anterior se trataron las funciones o métodos en Java, se desea también que en todos los programas a diseñar e implementar se tengan **métodos que hagan tareas específicas**. Como requisito adicional, toda la lectura de datos (uso de *Scanner*) y toda impresión de datos a la salida estándar (uso de *System.out*) debe hacerse en el método *main*; es decir, en los demás métodos no puede haber ni lectura ni impresión de datos.

Aunque todos los problemas de este Escenario se pueden resolver usando colecciones estándar del lenguaje como *TreeSet*, *ArrayList* y *TreeMap*, es posible que el estudiante conozca esta estructura; parte del trabajo aquí es intentar resolver el problema únicamente con arreglos de tamaño fijo. Posteriormente, se proponen estos ejercicios para ser resueltos con contenedores estándar de Java.

1. Hacer un seguimiento paso a paso al programa mostrado en los códigos 10 y 11, asumiendo que se lee el código estudiantil o número de cédula. Recomendación: usar una hoja de cálculo o papel y lápiz. Luego, correr el programa con la entrada que se hizo el seguimiento y verificar que todas las salidas correspondan a las detectadas durante este.

```
import java.util.Arrays;
import java.util.Scanner;
public class LoopsArraysAndMethods{
  public static void main (String[] args){
      Scanner sn = new Scanner(System.in);
      System.out.println("Let the game begins, digit your id:");
      int cc, arr[] = new int[countDigits(- (cc = (sn.nextInt()%1000000) ))], cnt[] = new int[10];
      for (int x = 0; x < arr.length; x++) {
            arr[arr.length-x-1] = cc%10; cc /
            = 10;
        }
        System.out.println("arr: "+Arrays.toString(arr));
        int i  = -1, j = 666, times = 0,k = 999, tmp;
        while( true ){
            ++i;
            if( i == arr.length-1 || i == 4)
                 break;
             if( i%2 != 0 )
                continue;
            for (j = arr.length-1,k=0; j >= 0 ; j--,++k){
                if( j%2 != 0 || k>=5)
                     continue;
                tmp = arr[i]+arr[j];
                System.out.println((++times)+" -> "+i+","+j+","+arr[i]+"+"+arr[j]+"="+tmp); do
                {
                        cnt[tmp%10]++;
                        tmp /= 10;
                }while(tmp>0);

            }
        }
        System.out.println("cnt: "+Arrays.toString(cnt));
        System.out.println("Game over man, game ooover!: "+i+","+j+","+k+","+times);
            sn.close();
    }
```

Código 10: seguimiento a programa paso a paso 1/2.

```
    public static int countDigits( long n ){
        int answer = 0;
        if( n < 0 )
            return countDigits(-n);
        while( n > 0 ){
            ++answer;
            n = n/10;
        }
        return answer;
    }
}
```

Código 11: seguimiento a programa paso a paso 2/2.

2. Hacer un seguimiento paso a paso al siguiente programa, asumiendo que se lee el código estudiantil o número de cédula. Recomendación: usar una hoja de calculo o papel y lápiz. Después, correr el programa con la entrada que se hizo el seguimiento y verificar que todas las salidas correspondan a las detectadas durante este.

```
import java.util.*;
public class ArraysAndMethdos{
    public static void main (String[] args) {
        Scanner sn = new Scanner(System.in);
        long id = sn.nextLong();
        int le = auxFunction(id,1);
        int s = auxFunction(id,2);
        System.out.println(le+" "+s);
        int arr[] = new int[le];
        int cnt[] = new int[10];
        long copy = id;
        for (int x = 0; x < arr.length; x++) {
            arr[arr.length-x-1] = (int) (id%10);
            id /= 10;
        }
        System.out.println("arr: "+Arrays.toString(arr));
        boolean f1[] = new boolean[le];
        boolean f2[] = new boolean[le];
        int index = le-1;
        id = copy;
        while(true){
            if( id == 0 )
                break;
            int r = (int) (id%10);
            ++cnt[r];
            f1[index] = r%2 == 0;
            id /= 10;
            --index;
        }
        for (int i = 0; i < le; i++){
            f2[i] = arr[i]%2==0;
        }
        System.out.println("f1: "+Arrays.toString(f1));
        System.out.println("f2: "+Arrays.toString(f2));
        int j = 0, sum = 0;
        id = copy;
        do{
            int r = (int) (id%10);
            ++cnt[r];
            sum += r;
            if( r != arr[le-j-1] || sum > s || f1[j] != f2[j] ){
                System.out.println("Ups, wrong.");
                break;
            }
            id /= 10;+
            +j;
        }while(j < le);
        System.out.println("cnt "+Arrays.toString(cnt));
        sn.close();
    }
```

Código 12: seguimiento a programa paso a paso 1/2.

```
    public static int auxFunction( long n , int option){
        if( n < 0 )
            return auxFunction(-n,option); int
        answer = 0;
        while( n > 0 ){
            if( option == 1 )
                ++answer;
            else
                answer += n%10;
            n = n/10;
        }
        return answer;
    }
}
```

Código 13: seguimiento a programa paso a paso 2/2.

3. Contador de dígitos. Diseñe e implemente un programa en Java que responda la siguiente pregunta: si quisiera escribir con lápiz y papel los números enteros desde el 0 hasta el *n* (*n* es un argumento que lee el programa), ¿cuántas veces escribiría cada dígito del 0 al 9?
   - Descripción de entrada: un número entero entre 0 y 10000.
   - Descripción de salida: 10 líneas, una para cada dígito del 0 al 9 indicando cuántas veces se escribiría el dígito al digitar los números del 0 al *n*.
   - Ejemplo de entrada:
     12
   - Ejemplo de salida:
     El dígito 0 se escribiría 2 veces.
     El dígito 1 se escribiría 5 veces.
     El dígito 2 se escribiría 2 veces.
     El dígito 3 se escribiría 1 veces.
     El dígito 4 se escribiría 1 veces.
     El dígito 5 se escribiría 1 veces.
     El dígito 6 se escribiría 1 veces.
     El dígito 7 se escribiría 1 veces.
     El dígito 8 se escribiría 1 veces.
     El dígito 9 se escribiría 1 veces.
4. Centro de masas. Dado un conjunto de puntos con masa en el plano, hacer un programa que lea estos datos y calcule y muestre el centro de masas de estos puntos. El centro de masas esta´ dado por la siguiente fo´rmula:

$$x_{cm} = \frac{\sum_{i=1}^{n} m_i * x_i}{\sum_{i=1}^{n} m_i}$$

$$y_{cm} = \frac{\sum_{i=1}^{n} m_i * y_i}{\sum_{i=1}^{n} m_i}$$

   - Descripción de entrada: un número entero *n* seguido de *n* triplas de números. Cada tripla es un punto con su masa, primero la coordenada *x*, luego la coordenada y y después la masa *m* para ese punto.

- Descripción de salida: dos números, la coordenada *x* del centro de masa y la coordenada y del centro de masa de los puntos leídos.
- Ejemplo de entrada:
  5
  10 10 2
  10 20 4
  10 30 8
  10 40 1
  10 50 16
- Ejemplo de salida:
  10 38.065

5. Espectro. Diseñar e implementar un programa que tome como entrada 5 números enteros positivos y produzca como salida un patrón como el que se muestra en el ejemplo.
   - Descripción de entrada: un número *n* entero positivo seguido de *n* números enteros no negativos. Todos entre 0 y 100.
   - Descripción de salida: el patrón según se muestra en el ejemplo. El primer número leído n será la cantidad de columnas de la matriz a imprimir. Los siguientes *n* números serán la cantidad de numerales que salen en cada columna. Así, en el ejemplo, en la primera columna hay 3 numerales, en la segunda un numeral, en la tercera 8 numerales (el mayor valor da la cantidad de filas), la penúltima columna muestra 4 numerales y en la última hay 2 numerales.
   - Ejemplo de entrada:
     5
     3 1 8 4 2
   - Ejemplo de salida:

```
..#..
..#..
..#..
..#..
..#..
..##.
#.##.
#.###
#####
```

Código 14: patrón a imprimir en el problema.

6. Trompo: diseñar e implementar un programa que tome como entrada que lea un número *n* y muestre en pantalla la figura del Código 15, dependiendo del valor de *n*.
   - Descripción de entrada: un número entero entre 0 y 7.
   - Descripción de salida: figura según el ejemplo para el valor leído *n*. Se puede ver que la primera fila es de dos *X* y aparece centrado respecto a la fila con mayor cantidad de *X*. La segunda fila tiene 4 *X*, la tercer 8 y así cada fila va aumentando según potencias de dos hasta la n-ésima fila. Después, el patrón se repite, pero al revés por $n - 1$ filas.
   - Ejemplo de entrada:
   - 5
   - Ejemplo de salida:

```
                    XX
                   XXXX
                 XXXXXXXX
           XXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
           XXXXXXXXXXXXXXXX
                 XXXXXXXX
                   XXXX
                    XX
```

Código 15: Figura

7. Palíndromas. Diseñe un programa que tome como entrada una cadena de texto e imprima y responda la pregunta: ¿cuántas subcadenas son palíndromas de longitud al menos 2 tiene la cadena leída. El programa debe mostrar todas las subcadenas palíndromas solicitadas en orden lexicográfico y la cantidad de ellas.
   - Descripción de entrada: una cadena de texto.
   - Descripción de salida: las cadenas de texto según lo descrito en el enunciado, de a una por línea, y el número indicando la cantidad de cadenas mostradas en una línea separada.
   - Ejemplo de entrada:
   - anitalavalatinaanaeslamejor
   - Ejemplo de salida:
     aa
     ala
     ala
     alavala ana
     anitalavalatina
     ava
     italavalati
     laval
     naan
     nitalavalatin
     talavalat
     12
8. Diseñar e implementar un programa que tome como entrada un número entero *n* y seguido a esto lea *n* números enteros. El programa debe mostrar los números leídos sin repeticiones y ordenados ascendentemente.
   - Descripción de entrada: un número *n* entero, seguido de n números enteros, todos mayores o iguales que 0 y menores o iguales a 1000.
   - Descripción de salida: los elementos leídos, sin repeticiones y ordenados ascendentemente.
   - Ejemplo de entrada:
     14
     10 29 5 6 29 10 10 10 6 6 6 5 5 5
   - Ejemplo de salida:
     5 6 10 29
9. Diseñar e implementar un programa que tome como entrada un número entero *n* y seguido a esto lea *n* números enteros y diga cuántas veces está cada entero leído en la lista de enteros leídos. Adicionalmente, debe mostrar los números ordenados por frecuencia, es decir, primero el que más veces aparece en la entrada; después, el segundo más frecuente; luego, el tercero y así sucesivamente.

- Descripción de entrada: un número *n* entero, seguido de *n* números enteros todos mayores o iguales que
  0 y menores o iguales a 1000.
- Descripción de salida: la cantidad de veces que aparece cada número. Los números deben aparecer en orden descendente respecto a la cantidad de veces que se presentan.
- Ejemplo de entrada:
  13
  10 29 5 6 29 10 2 2 2 10 2 10 2
- Ejemplo de salida:
  El 2 aparece 5 veces.
  El 10 aparece 4 veces.
  El 29 aparece 2 veces.
  El 6 aparece una vez.
  El 5 aparece una vez.

10. Frecuencias: diseñar e implementar un programa que tome como entrada un número entero *n* y luego de esto lea *n* números enteros; después debe mostrar una tabla de frecuencias de los números leídos usando asteriscos. Deben estar ordenados en la salida respecto a la frecuencia; primero los que más veces aparecen en la entrada.
    - Descripción de entrada: un número *n* entero, seguido de *n* números enteros, todos mayores o iguales que 0 y menores o iguales a 1000.
    - Descripción de salida: la tabla de asteriscos según el ejemplo.
    - Ejemplo de entrada:
      13
      10 29 5 6 29 10 2 2 2 10 2 10 2
    - Ejemplo de salida:
      2 *****
      10 ****
      29 **
      6 *
      5 *
11. Frecuencias: diseñar e implementar un programa que tome como entrada un número entero *n* y seguido a esto lea *n* números enteros; después debe mostrar una tabla de frecuencias de los números leídos usando asteriscos *, los cuales deben estar ordenados en la salida respecto a la frecuencia. La cantidad de asteriscos mostrados por número debe ser la mínima posible, siempre y cuando se conserve la proporción respecto a la cantidad de números leída. Por ejemplo, en la entrada mostrada, aparecen 4 asteriscos para el número 10, aunque el número 10 aparece 12 veces. Esto es porque la cantidad de asteriscos por número es proporcional a la cantidad de veces que cada número se leyó en la entrada. En el ejemplo, la cantidad de números es 3 multiplicado por la cantidad de asteriscos: 12 veces el número 10, 6 veces el número 29, tres veces el número 6 y 3 veces el número 5 (12+6+3+3=24).
    - Descripcio´n de entrada: un número *n* entero, seguido de *n* números enteros, todos mayores o iguales que 0 y menores o iguales a 1000.
    - Descripción de salida: la tabla de asteriscos según el ejemplo.
    - Ejemplo de entrada:
      24
      10 29 5 6 29 10 10 10 10 29 5 6 29 10 10 10 10 29 5 6 29 10 10 10
    - Ejemplo de salida:

10 ****
29 **
6 *
5 *

12. Se desea hacer un programa que tome como entrada los datos de estudiantes (nombres completos, números de documento) y una serie de notas; debe producir una lista Estudiante-Promedio, ordenada de mejor a peor promedio de notas, y mostrar el resultado a manera de tabla con tres columnas: nombre, cédula y promedio. El formato del archivo de entrada es el siguiente:
    - En la primera línea del archivo de entrada habrá un número entero positivo *n*; este será la cantidad de líneas que siguen. Cada línea representará la información de un estudiante y tendrá el siguiente formato: nombre completo - número de documento - serie de notas entre 0 y 5 separadas por espacios.
    - Descripción de salida: la lista según lo solicitado.
    - Ejemplo de entrada:
      4 Rafael Alberto Velasquez-889993893-4 3 4 4.5 4 2 1
      Jose Figueroa-2334234-4 3 1 3 5
      Diego Castilla-232234-3 4 1 2 3 3 2 1 2.3 2.3
      Alberto Baron-234213-3 3 1
    - Ejemplo de salida:
      Alberto Baron 234213 4.33333
      Rafael Alberto Velasquez 889993893 3.21429
      Jose Figueroa 2334234 3.2
      Diego Castilla 232234 2.36
13. Número de Fibonacci. Para este programa se recomienda implementar la función recursiva. Optimizar usando memoization como se mostró en este escenario.:

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-1) + f(n-2) & \text{en otro caso.} \end{cases}$$

    - Descripción de entrada: un número entero positivo entre 0 y 90.
    - Descripción de salida: un entero positivo, el enésimo número de fibonacci (The OEIS Foundation, 2018).
    - Ejemplo de entrada:
      0
    - Ejemplo de salida:
      1
    - Ejemplo de entrada 2:
      1
    - Ejemplo de salida 2:
      1
    - Ejemplo de entrada 3:
      2
    - Ejemplo de salida 3:
      2
    - Ejemplo de entrada 4:
      40
    - Ejemplo de salida 4:
      102334155

Referencias

Cormen, T., Leiserson, C., Rivest, R. y Stein, C. (2009). *Introduction to Algorithms.* Cambridge, MA, EUA: The MIT Press.

Guerequeta, R. y Vallecillo, A. (2000). *Técnicas de Diseño de Algoritmos.* Málaga, España: Servicio de Publicaciones de la Universidad de Malaga.

Oracle, C. (1995). Arrays in Java. Recuperado el 30 de abril de 2018, desde https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html

The OEIS Foundation. (2018). *The On-Line Encyclopedia of Integer Sequences.* Recuperado de https://oeis.org/A000045

INFORMACIÓN TÉCNICA

Ingeniería y Ciencias Básicas

Módulo: Conceptos Fundamentales de Programación
Unidad 2: Funciones y arreglos
Escenario 4: Arreglos de tamaño fijo

Autor: Rafael Niquefa

Asesor Pedagógico: Angie Laitón
Diseñador Gráfico: Rafael Niquefa
Asistente: Alejandra Morales

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.